TransitPulse MBTA

Tài liệu thiết kế hệ thống dữ liệu giao thông

Phiên bản 1.0 · Ngày 05 tháng 10 năm 2026 · Trạng thái Thiết kế trước triển khai

TransitPulse MBTA là hệ thống thu thập, xử lý và phục vụ dữ liệu xe buýt của Massachusetts Bay Transportation Authority tại khu vực Boston. Sản phẩm giúp người dùng xem vị trí xe có dữ liệu mới, dự báo đến trạm, độ trễ dự kiến và cảnh báo dịch vụ; đồng thời cho phép nhóm phát triển chứng minh dữ liệu đã đi qua pipeline có kiểm soát chất lượng và có khả năng phục hồi.

Tài liệu này chuyển đặc tả xây dựng dự án thành thiết kế nghiệp vụ và kỹ thuật để phân công, triển khai, kiểm thử và nghiệm thu. Công nghệ trọng tâm của môn học là **Apache Cassandra cho quản lý dữ liệu** và **Apache Spark Structured Streaming cho xử lý dữ liệu**. Kafka đảm nhiệm vận chuyển sự kiện; FastAPI và Streamlit đưa dữ liệu đã xử lý tới ứng dụng.

MVP theo dõi năm tuyến bus được xác minh trong GTFS tại thời điểm triển khai. Cả chế độ trực tiếp, phát lại bản ghi và dữ liệu kiểm thử đều dùng cùng đường xử lý Kafka → Spark → Cassandra → API. Mỗi chế độ có namespace và nhãn nguồn riêng. Ứng dụng không đọc trực tiếp feed MBTA để bỏ qua pipeline.

Đây là **thiết kế**, chưa phải báo cáo hệ thống đã chạy. Các ngưỡng hiệu năng, cấu hình tài nguyên và kết quả kiểm thử trong tài liệu là mục tiêu hoặc giá trị kỳ vọng của fixture. Chưa có số liệu benchmark hay biên bản nghiệm thu thực tế.

Thông tin kiểm soát	Giá trị
Tên dự án	TransitPulse MBTA
Loại tài liệu	System Design Document và cơ sở nghiệm thu
Nền tảng đặc tả	TransitPulse MBTA Full Build Prompt VI
Nguồn dữ liệu ngoài	MBTA GTFS static và ba feed GTFS Realtime JSON
Sản phẩm dự kiến	Pipeline streaming, dữ liệu phục vụ, API, dashboard và bộ bằng chứng
Người sử dụng tài liệu	Nhóm triển khai, giảng viên và người đánh giá
Cần cập nhật sau triển khai	Phiên bản thực tế, route IDs, benchmark, test report và giới hạn quan sát được

Mục lục

Các phần được tổ chức theo nghiệp vụ, dữ liệu, xử lý, phục vụ và kiểm chứng. Số trang theo bản PDF đi kèm; Word có thể thay đổi phân trang khi chỉnh sửa.

Nội dung	Trang
1 Nghiệp vụ và phạm vi	3
2 Yêu cầu chức năng	4
3 Yêu cầu phi chức năng	5
4 Nguồn dữ liệu và hợp đồng nguồn	6
5 Kiến trúc hệ thống	7
6 Mô hình nghiệp vụ GTFS	8
7 Run snapshot và định danh dữ liệu	9
8 Thời gian lịch trình và đồng hồ	10
9 Thu thập raw và durable outbox	11
10 Kafka và hợp đồng envelope	12
11 Xử lý Spark và publish generation	13
12 Snapshot semantics và phục hồi	14
13 Mô hình Cassandra metadata và static	15
14 Mô hình Cassandra serving và retention	16
15 Định nghĩa vị trí và dự báo	17
16 KPI độ trễ dự kiến và ví dụ	18
17 Thiết kế API và truy vấn	19
18 Response contract và lỗi	20
19 Thiết kế dashboard	21
20 Chất lượng dữ liệu và observability	22
21 Replay synthetic và demo offline	23
22 Triển khai cấu hình và an toàn	24
23 Runbook vận hành và xử lý sự cố	25
24 Kế hoạch kiểm thử và oracle	26
25 Benchmark và bằng chứng thành công	27
26 Nghiệm thu và truy vết yêu cầu	28
27 Tổ chức triển khai và bàn giao	29
28 Phụ lục CQL và state machines	30
29 Nguồn tham chiếu và thuật ngữ	31

1 Nghiệp vụ và phạm vi

Vấn đề cần giải quyết

Feed giao thông liên tục thay đổi, có thể thiếu trường, bị trễ hoặc tạm ngừng cập nhật. Việc hiển thị response mới nhất không đủ để biết một tọa độ có mới hay không, một dự báo có đáng dùng hay không và dữ liệu đã được xử lý đầy đủ hay chưa. TransitPulse giải quyết vấn đề này bằng dữ liệu có phiên bản, thời gian nguồn rõ ràng, kiểm tra chất lượng và truy vết từ raw tới API.

Phần của đề tài	Công việc của dự án	Bằng chứng cần có
Business và domain data	Xác định tuyến, trạm, chuyến, xe, dự báo và cảnh báo; mô tả câu hỏi người dùng	Use cases, data dictionary, scope đã xác minh
Data engineering	Thu thập, lưu raw, vận chuyển, chuẩn hóa, nối GTFS, khử trùng, lưu Cassandra	Logs, lineage, bảng dữ liệu, test phục hồi và benchmark
Application	Bản đồ, ETA, KPI tuyến, cảnh báo và trang chất lượng dữ liệu	API responses và demo dashboard dùng dữ liệu đã xử lý

Người dùng và tình huống sử dụng

Vai trò	Nhu cầu	Tình huống chính
Hành khách	Biết xe và dự báo đến trạm có còn mới	Chọn tuyến và trạm, xem ETA, nhãn chất lượng, cảnh báo
Người theo dõi vận hành	Nhìn mức độ chậm dự kiến của các chuyến có dữ liệu đủ điều kiện	Xem KPI tuyến, mẫu số và thời điểm tính
Data engineer	Phát hiện lỗi nguồn, lag, duplicate và lỗi sink	Xem pipeline status, quality reasons, raw reference
Người đánh giá	Kiểm chứng công nghệ và tính đúng của kết quả	Phát lại fixture, đối chiếu đầu ra, gây lỗi rồi khởi động lại

Ranh giới MVP

MVP gồm năm tuyến `route_type=3`, ba feed realtime, một GTFS version cho mỗi run, recording mục tiêu hai giờ, lịch sử quan sát và dashboard năm trang. Chỉ mở rộng lên mười tuyến hoặc recording 24–72 giờ sau khi đo dung lượng và độ trễ. Danh sách route IDs là cấu hình được xác minh, chưa được gán trước trong tài liệu này.

MVP chưa xây mô hình dự báo ETA, tối ưu hành trình, đo số hành khách, đánh giá độ chính xác dự báo hay triển khai nhiều vùng. Độ trễ dự kiến không phải độ trễ thực tế đã đo tại trạm. Cassandra và Kafka một node phục vụ bài môn học; cấu hình này không có bảo đảm high availability.

2 Yêu cầu chức năng

Mỗi yêu cầu có mã dùng xuyên suốt thiết kế và kiểm thử. Đầu ra phải quan sát được qua file, API hoặc dashboard; hoàn thành giao diện mà pipeline chưa thực hiện biến đổi không được coi là hoàn thành yêu cầu.

Mã	Yêu cầu và điều kiện hoàn thành
FR01	Nhập GTFS ZIP, giữ ID dạng string, xác minh khóa và scope, tạo dimensions Parquet cùng metadata version bất biến.
FR02	Thu thập độc lập ba feed JSON theo lịch cấu hình; lưu raw và metadata trước khi publish; timeout không tạo empty snapshot giả.
FR03	Vận chuyển snapshot qua Kafka có schema, checksum và ID ổn định; outbox tiếp tục pending khi collector khởi động lại.
FR04	Spark thực hiện parse, validate, explode, nối GTFS, dedup và tạo serving views; lỗi dữ liệu có quality reason và raw reference.
FR05	Phục vụ vị trí theo current generation; phân biệt fresh, stale, unknown và invalid; không cộng số xe qua nhiều snapshot.
FR06	Phục vụ dự báo đến hoặc rời trạm, xử lý uncertainty MBTA; tính predicted delay khi xác định được đúng lịch và trip instance.
FR07	Tính KPI tuyến trên tối đa một target stop cho mỗi trip instance đủ điều kiện; công bố numerator, denominator, exclusions và as of.
FR08	Đánh giá service alert selectors và thời hạn; phân biệt cảnh báo MBTA với cảnh báo chất lượng pipeline.
FR09	Cung cấp lịch sử xe theo thời gian có paging; ghi curated Parquet và tạo phân tích lịch sử bằng Spark batch.
FR10	Publish immutable generation; API chỉ đọc generation READY; duplicate, late snapshot và retry không làm tăng KPI hoặc rollback pointer.
FR11	Phát lại recording đúng static version, hỗ trợ 1x, 10x, 50x, pause và resume; source clock tách wall clock; tách run outputs.
FR12	Cung cấp API và dashboard, quality, progress, lỗi và phiên bản dữ liệu; thực hiện golden tests, fault tests và benchmark có bằng chứng.

Kịch bản người dùng xuyên suốt

Người dùng chọn run và tuyến. API đọc metadata để xác định scope và GTFS version, sau đó đọc pointer của từng feed một lần. Dashboard hiển thị xe và dự báo theo các generation tương ứng, ghi rõ thời điểm và chất lượng. Khi feed ngừng, tuổi quan sát vẫn tăng theo application clock; xe cũ chuyển sang stale thay vì tiếp tục hiển thị như dữ liệu mới.

Khi người đánh giá chuyển sang replay, dashboard luôn hiện nhãn REPLAY và clock lịch sử. Việc phát lại một snapshot hai lần phải cho cùng trạng thái phục vụ. Khi dừng Spark giữa lúc ghi sink rồi khởi động lại với checkpoint cũ, thế hệ chưa hoàn tất không được xuất hiện qua API và kết quả sau phục hồi phải khớp fixture.

3 Yêu cầu phi chức năng

Mã	Tiêu chí	Mục tiêu thiết kế và cách kiểm tra
NFR01	Tính đúng	Golden fixture có kết quả xác định; so API, Cassandra và report với oracle độc lập.
NFR02	Phục hồi	At least once delivery với ghi và publish idempotent; restart cùng checkpoint không cộng trùng trong các fault scenarios được kiểm thử.
NFR03	Độ trễ	Mục tiêu p95 dưới 10 giây từ enqueue thành công tới first visibility ở cấu hình tham chiếu; chưa đo.
NFR04	Dung lượng có giới hạn	Selected envelope tối đa 4 MiB; Kafka retention mục tiêu 6 giờ; raw budget tham chiếu 2 GiB; đo thực tế.
NFR05	Truy vết	Mỗi view truy về run, scope, static version, snapshot, checksum và raw reference.
NFR06	Tính tái lập	Pin dependencies sau preflight, giữ static và recording; fixture seed cố định; cùng input và policy cho cùng logical output.
NFR07	Khả năng quan sát	Có metrics theo đơn vị, structured logs và thời điểm đo; lỗi nguồn khác lỗi processing khác lỗi serving.
NFR08	Giới hạn truy vấn	Query Cassandra theo partition; history tối đa 24 giờ mỗi request; paging và row limits; không dùng ALLOW FILTERING.
NFR09	An toàn demo	Host ports bind localhost; CORS allowlist; không public Kafka hoặc Cassandra; raw path được kiểm soát bởi cấu hình.
NFR10	Demo offline	Chạy cùng pipeline với assets, images, wheels, JARs, static và recording đã chuẩn bị; kiểm tra thực tế khi tắt mạng.

Phân biệt độ trễ và tuổi nguồn

Polling 30 giây là chu kỳ thu thập của dự án. Target p95 dưới 10 giây bắt đầu từ lúc enqueue thành công tới lúc generation lần đầu visible, vì vậy không bao gồm thời gian chờ lần poll tiếp theo hoặc thời gian MBTA chưa công bố dữ liệu. Tuổi quan sát được tính bằng clock nghiệp vụ trừ thời điểm nguồn đo vị trí, là phép đo khác.

Cấu hình tài nguyên tham chiếu

Môi trường triển khai dự kiến có 4 CPU và khoảng 16 GiB RAM, Spark `local[2]`, một Kafka broker KRaft và một Cassandra node với replication factor 1. Đây là cấu hình đầu vào cho thử nghiệm, chưa phải sizing đã được xác nhận. Nếu thiếu tài nguyên, nhóm giảm scope hoặc recording sau khi đo và ghi lại thay đổi; không bỏ qua thành phần công nghệ bắt buộc để báo đạt.

4 Nguồn dữ liệu và hợp đồng nguồn

Chỉ sử dụng MBTA mục 1 trong đặc tả. Các URL là điểm truy cập; nội dung tại cùng URL có thể thay đổi nên version phải dựa trên dữ liệu tải về. Schema và provider semantics đối chiếu tài liệu MBTA và GTFS chính thức [S1–S4].

Nguồn	URL	Nội dung và vai trò
GTFS static	https://cdn.mbta.com/MBTA_GTFS.zip	Lịch, tuyến, trạm, chuyến, shape, calendar và timezone
Vehicle Positions	https://cdn.mbta.com/realtime/VehiclePositions.json	Membership xe, tọa độ, descriptor và thời điểm quan sát
Trip Updates	https://cdn.mbta.com/realtime/TripUpdates.json	Stop time predictions, delay, uncertainty và trip relationship
Service Alerts	https://cdn.mbta.com/realtime/Alerts.json	Active periods, selectors, header, description và ảnh hưởng

Tệp static và kiểm tra tối thiểu

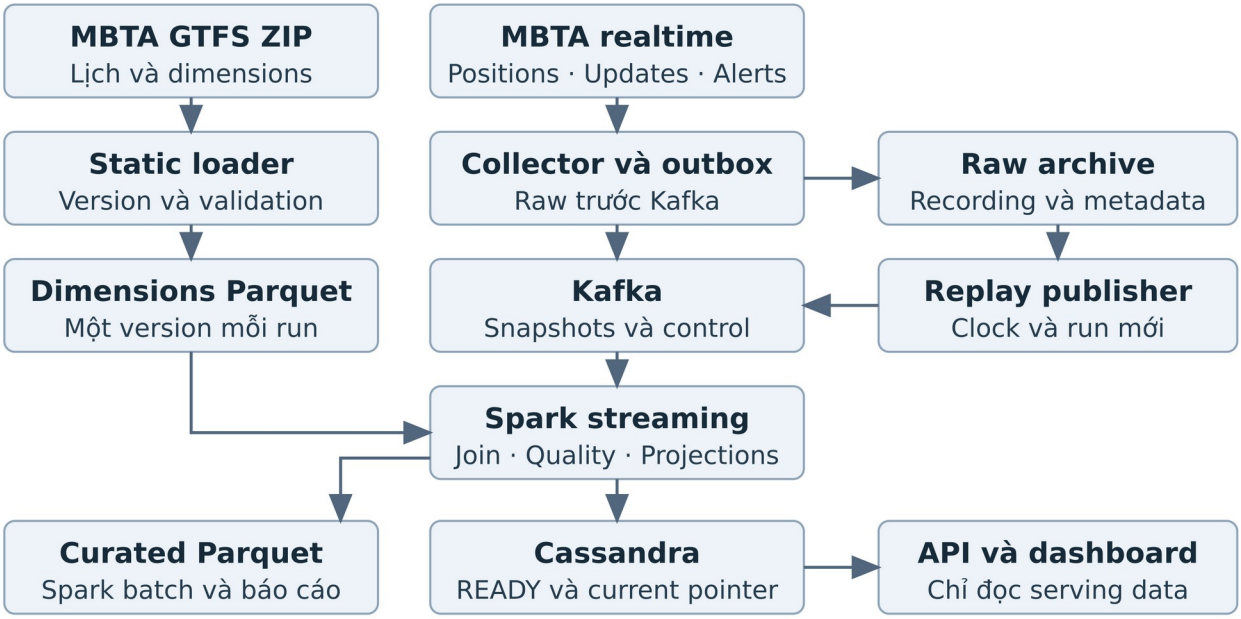
Tệp	Khóa và nội dung sử dụng
agency.txt	agency_id, agency_timezone; cơ sở chuyển lịch sang UTC
routes.txt	route_id, agency_id, route_type, tên tuyến; chọn bus type 3
trips.txt	trip_id, route_id, service_id, direction_id, shape_id
stops.txt	stop_id, tên, tọa độ và quan hệ parent khi có
stop_times.txt	trip_id và stop_sequence, stop_id, arrival_time, departure_time
calendar.txt	service_id, ngày trong tuần và khoảng hiệu lực khi có
calendar_dates.txt	Ngoại lệ thêm hoặc bỏ service theo ngày
shapes.txt	shape_id và sequence, tọa độ các điểm đường
feed_info.txt	Metadata feed và provider feed_version khi có

Preflight nguồn

Trước triển khai sâu, tải một ZIP và mẫu ba feed; ghi HTTP status, checksum, timestamp, incrementality, entity counts, trường thiếu và kích thước. Xác minh năm route IDs tồn tại và có realtime phù hợp; kiểm tra file optional hoặc calendar exceptions theo chuẩn thay vì giả định mọi tệp luôn có dữ liệu. Tạo `source_probe_report.json` và cấu hình scope được chốt.

Chưa có bản ghi live được tạo trong tài liệu này. Không khẳng định các URL có SLA hoặc dữ liệu hoàn chỉnh tại mọi thời điểm. Không tự chuyển sang MBTA V3 API, LAMP hay nguồn khác khi một feed lỗi; thay đổi nguồn cần sửa hợp đồng dữ liệu và nghiệm thu tương ứng.

5 Kiến trúc hệ thống



Hình 1 Kiến trúc và các đường dữ liệu của TransitPulse MBTA

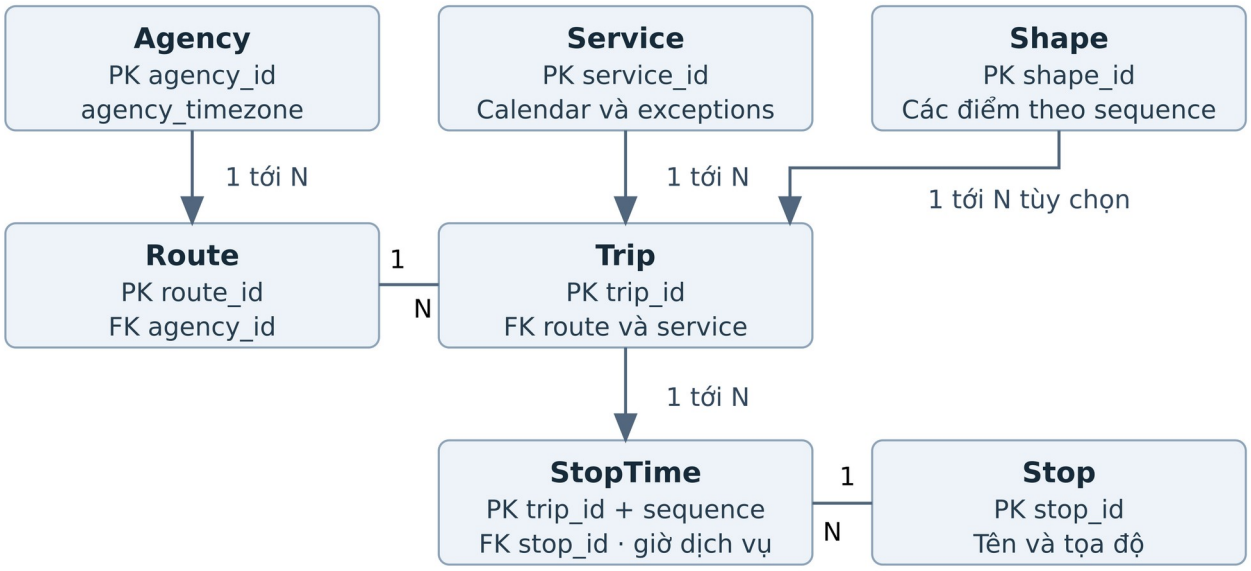
Trách nhiệm thành phần

Thành phần	Trách nhiệm chính
Static loader	Version GTFS, validate và materialize dimensions Parquet cùng catalog Cassandra
Collector và replay publisher	Lấy hoặc đọc raw, quản lý outbox, lọc scope và publish envelope; không tính KPI
Kafka	Lưu sự kiện vận chuyển và control theo run, feed; cung cấp offsets và retention
Spark	Chuẩn hóa, nối dimensions, quality, dedup, projections, lịch sử và KPI
Cassandra	Các bảng phục vụ thiết kế theo query; manifest và current pointer
FastAPI	Bounded reads, freshness tại request, response contract và lỗi có cấu trúc
Streamlit	Chọn run, hiển thị nguồn, thời gian, dữ liệu và trạng thái bằng API

Raw archive là đường audit và replay, không phải đường dashboard đọc tắt. Curated Parquet là đầu vào phân tích lịch sử bằng Spark batch. Một processing instance chịu trách nhiệm một run; multi run đồng thời là bước mở rộng sau khi đo tài nguyên.

Không có transaction toàn hệ thống giữa Kafka, filesystem và Cassandra. Thiết kế kiểm soát visibility bằng generation READY và pointer, kết hợp retry idempotent để phục hồi. Chi tiết giao thức ở các phần 11 và 12.

6 Mô hình nghiệp vụ GTFS



Quan hệ nghiệp vụ GTFS · Một stop có thể xuất hiện nhiều lần trong trip

Hình 2 Quan hệ nghiệp vụ của dữ liệu GTFS static

Sơ đồ thể hiện quan hệ logic của GTFS, không phải foreign keys trong Cassandra. `Service` là khái niệm tổng hợp từ calendar và calendar exceptions. `Shape` là optional đối với chuyến khi nguồn không có shape; một route có nhiều trips, chiều và pattern.

Quy tắc cấu trúc

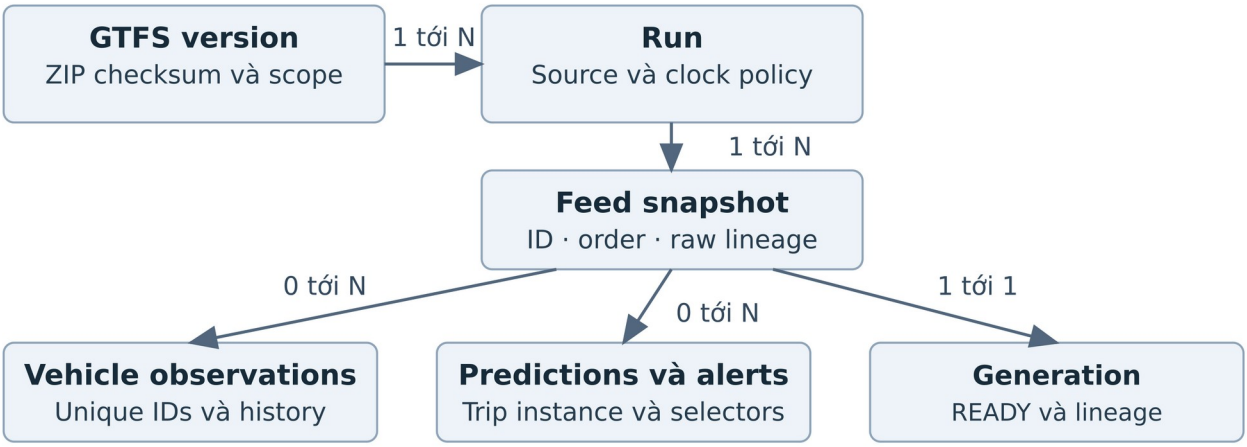
- Giữ mọi ID là string, kể cả khi giá trị trông như số. Không đổi ID rồi làm mất số 0 đầu hoặc ký tự của nguồn.
- Stop occurrence dùng `trip_id + stop_sequence`. Một trip có thể lặp `stop_id`, nên stop_id một mình chưa đủ xác định sự kiện.
- `stops_by_route` trả tập trạm liên quan tuyến. Thứ tự trạm được lấy theo trip; không ghép mọi stop của route thành một thứ tự giả.
- Geometry lưu theo shape, không nối các shape khác chiều thành một polyline. Shape points được sắp theo sequence.
- Uniqueness và các references cần thiết phải được kiểm tra trước khi dùng version cho một run; invalid static không được âm thầm làm nền tính delay.

Sản phẩm static loader

`gtfs_version` nội bộ là SHA 256 của ZIP. Metadata giữ provider feed_version nếu có, URL, tải lúc nào, parser version, timezone, scope và counts. Raw ZIP và dimensions giữ bất biến. Một run pin một version; tải ZIP mới tạo version mới và run mới thay vì thay dimensions trong run đang chạy.

Spark đọc dimensions Parquet đã chuẩn hóa. Cassandra giữ catalog theo query cho API. Quan hệ đầy đủ trips, service và lịch không cần được mô phỏng bằng joins Cassandra tại thời điểm phục vụ.

7 Run snapshot và định danh dữ liệu



Hình 3 Quan hệ logical giữa run snapshot và các kết quả xử lý

Định danh	Quy tắc và mục đích
run_id	UUID tạo một lần cho lần chạy; namespace của Kafka envelope, checkpoint, raw và serving
scope_id	Hash của danh sách route đã sắp cùng scope policy; phân biệt phạm vi xử lý
gtfs_version	SHA 256 ZIP; pin một version cho một run
capture_id	Định danh lần HTTP capture; phục vụ audit dù payload lặp
snapshot_id	Hash ổn định của run, scope, feed, header và selected logical payload; không chứa retry UUID hoặc published_at
observation_id	Hash vehicle, trip descriptor, thời điểm quan sát và position payload; cùng quan sát poll lại không sinh lịch sử mới
prediction_id	Trip instance, stop occurrence và loại arrival hoặc departure; payload version nằm trong snapshot
error_id	Hash reference và reason; giúp đối chiếu DLQ khi delivery bị lặp

Bất biến dữ liệu

Mọi output có run_id và gtfs_version. `source_type` nhận `live`, `replay` hoặc `synthetic`, không đổi ngầm trong một run. Synthetic fixture có scenario và seed; không dùng ID synthetic để tuyên bố là route hoặc xe MBTA thật.

Thiếu observation timestamp tạo `snapshot_state_sample`, dùng feed header làm effective event time và ghi `event_time_origin=feed_header`, `freshness_known=false`. ID và history key cùng dùng mốc hiệu dụng đó, không dùng fetch hoặc processing time để tạo quan sát mới mỗi lần poll. Record này không được tính như vị trí có tuổi đã biết hoặc dùng cho derived speed chính xác.

Hash snapshot dùng canonical serialization đã chốt, gồm optional fields và enum. Retry cùng logical payload cho cùng hash. Raw checksum tính trên bytes nguồn, khác hash của selected snapshot.

8 Thời gian lịch trình và đồng hồ

Trường thời gian	Ý nghĩa và clock
observation_time	Thời điểm nguồn đo một vị trí xe; source time, có thể thiếu
feed_timestamp	Thời điểm header của feed; không chứng minh xe vừa được đo
original_fetched_at	Wall UTC tại lần capture gốc; giữ nguyên khi replay
published_at	Wall UTC khi enqueue trong run hiện tại; dùng đo transport latency
processed_at và view_published_at	Wall UTC khi xử lý và first visibility
application_clock	Wall UTC cho live; historical UTC theo tick cho replay
served_at	Wall UTC khi API trả response; không phải first visibility

Chuyển lịch GTFS sang UTC

Giờ `HH:MM:SS` được đổi thành số giây từ service day; `25:10:00` là hợp lệ. Dùng timezone của agency và nguyên tắc GTFS local noon trừ 12 giờ [S3].

```
gtfs_seconds = HH * 3600 + MM * 60 + SS
service_anchor_utc = UTC(local_noon(service_date, agency_timezone)) - 12 hours
scheduled_event_utc = service_anchor_utc + gtfs_seconds
```

Đối chiếu arrival với arrival, departure với departure. Kiểm tra MM và SS trong 0–59, calendar hiệu lực và exception trước khi nhận scheduled timestamp. Test cần bao phủ giờ lớn hơn 24, chuyến qua đêm, DST mùa xuân và mùa thu. Cộng giờ vào naive local midnight không đủ cho các trường hợp này.

Xác định trip instance

Ưu tiên `start_date` từ TripDescriptor; giữ thêm start_time, route_id và direction_id khi có. Nếu nguồn thiếu date, xét các service date ứng viên phù hợp với calendar và chuyến qua đêm. Chỉ resolve khi có một đáp án hợp lệ; nếu mơ hồ, trả `schedule_resolution=ambiguous`, vẫn có thể hiển thị giờ dự báo nguồn nhưng loại khỏi KPI delay.

Join stop bằng stop_sequence khi có. Nếu chỉ có stop_id, phải kiểm tra nó có duy nhất trong trip hay không. Cancelled hoặc chuyến mới không khớp lịch được giữ đúng relationship; không ép chúng vào scheduled trip để tạo delay giả.

Sai lệch đồng hồ

Tolerance timestamp tương lai tham chiếu là 5 giây, cấu hình và công bố trong run policy. Sai lệch nhỏ được giữ có warning và tuổi hiển thị không âm; lớn hơn ngưỡng bị loại khỏi tập fresh. Replay chỉ dùng source clock để đánh giá tuổi; late snapshot không được kéo clock lùi.

9 Thu thập raw và durable outbox

Collector flow

1. Scheduler lấy mỗi feed mặc định 30 giây, với timeout, retry hữu hạn, backoff và jitter. Một feed lỗi không chặn feed khác; tôn trọng Retry After và conditional GET nếu nguồn hỗ trợ.
2. Ghi bytes response gốc cùng metadata vào file tạm dưới namespace run và feed; đóng file rồi atomic rename. Raw lưu gzip JSON, checksum và capture time.
3. Parse đủ để xác định header, scope và selected entities. Không tính predicted delay hoặc route KPI trong collector.
4. Tạo envelope và durable pending outbox. Chỉ publish sau khi raw và outbox đã bền vững; message quá lớn lưu raw và reason thay vì truncate.
5. Gửi Kafka, đợi broker ACK, rồi đánh dấu published trong outbox. Khi restart, tiếp tục pending với cùng snapshot ID.

Nếu chết sau ACK và trước local mark, collector có thể gửi lại. Đây là delivery lặp hợp lệ của thiết kế; downstream chịu trách nhiệm idempotency. Filesystem và Kafka không được xem là một transaction chung.

Scope và snapshot

Vehicle Positions và Trip Updates ưu tiên route_id trong descriptor, nếu thiếu thì nối trip vào static có ghi lineage. Không resolve được scope thì giữ raw và reason. Alerts cần đánh giá đầy đủ selector agency, route, route type, trip và stop; thiếu route_id không có nghĩa alert áp dụng toàn bộ hệ thống.

HTTP 200 với FULL DATASET hợp lệ và zero selected entities là empty snapshot có ý nghĩa. Timeout, response lỗi, malformed JSON hoặc header invalid không được biến thành empty snapshot. HTTP 304 không tạo quan sát mới; collector cập nhật trạng thái request riêng với trạng thái dữ liệu nguồn.

Disk policy

Raw paths có dạng run/feed/ngày UTC/capture, được sinh từ root cấu hình; payload không tự chọn đường file. Manifest trỏ static version và raw checksum để replay không ghép sai lịch. Recording mục tiêu hai giờ và budget 2 GiB là tham chiếu cần đo. Vượt budget hoặc disk full phải dừng ghi và publish theo fail policy, báo lỗi rõ ràng. Tự động xóa raw mặc định tắt; retention chỉ xóa file thuộc manifest dự án và phải có dry run.

10 Kafka và hợp đồng envelope

Topic	Payload và vai trò
mbta.vehicle_positions.snapshots.v1	Selected snapshot vị trí xe
mbta.trip_updates.snapshots.v1	Selected snapshot dự báo chuyến
mbta.alerts.snapshots.v1	Selected snapshot cảnh báo dịch vụ
mbta.control.v1	Run lifecycle, collector status và replay clock ticks
mbta.dlq.v1	Record lỗi có reason, reference và excerpt có giới hạn

Envelope bắt buộc

Nhóm trường	Nội dung
Version và namespace	schema_version, run_id, scope_id, source_type, feed_type, gtfs_version
Định danh và lineage	snapshot_id, capture_id, source_url, raw_path, raw_sha256
Thời gian	feed_timestamp epoch seconds UTC, original_fetched_at, published_at ISO UTC
Accounting	source_entity_count, selected_entity_count
Payload	header và entities giữ semantics của nguồn
Kiểm thử	scenario_id nullable; metadata generator nằm trong recording hoặc run manifest

Một selected snapshot là một Kafka message, key `run_id|feed_type`, tối đa 4 MiB sau serialization envelope. Không đưa full response ngoài scope vào Kafka với kích thước không giới hạn. Header và selected entities phải đủ để downstream áp dụng full snapshot semantics.

Transport configuration

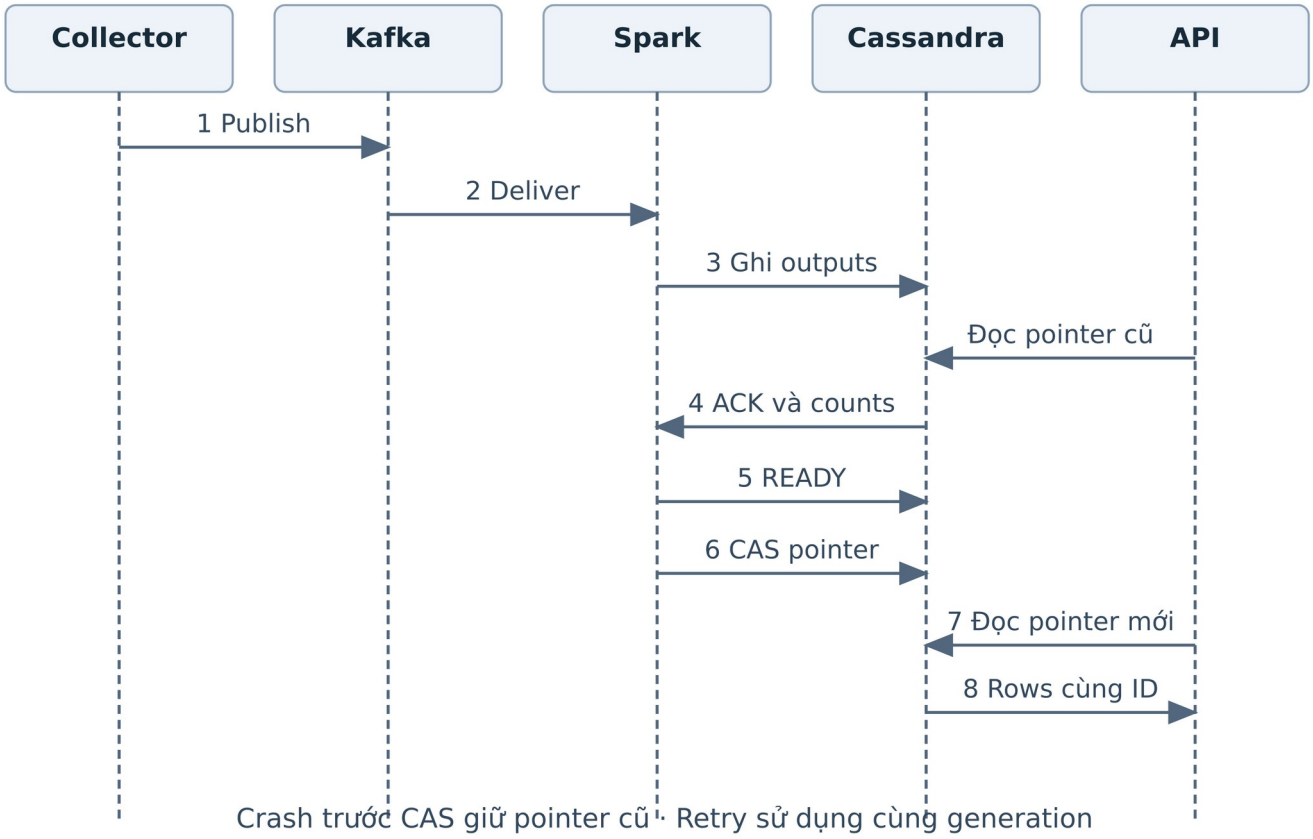
MVP dùng một partition mỗi topic, một broker và RF1. Producer `acks=all`, idempotence khi client và cấu hình hỗ trợ; tùy chọn này không tạo exactly once với Cassandra. Input retention mục tiêu sáu giờ và byte budget cần được cấu hình theo partition.

Preflight phải đo serialized size và phối hợp producer request size, broker và topic message size, consumer fetch size cùng Spark Kafka options. Ngưỡng chấp nhận payload là 4 MiB; transport limits cần phần dư cho protocol overhead. Gửi thử gần giới hạn, đúng giới hạn và vượt giới hạn. DLQ không chứa lại body khổng lồ; raw reference giữ dữ liệu đầy đủ.

Spark checkpoint mới đọc earliest trong retention rồi filter run; restart theo offsets trong checkpoint. Không dùng cùng kafka.group.id cho nhiều query và không che mất offsets bằng failOnDataLoss false [S5–S6].

11 Xử lý Spark và publish generation

Spark Structured Streaming đọc ba topic và control, dùng schema tường minh, trigger 5 giây. DataFrame transformations thực hiện parse, explode, chuẩn hóa, validation, join dimensions, dedup và group theo run/feed/snapshot. Không collect toàn bộ dataset về driver để thay Spark bằng vòng lặp Python.



Hình 4 Trình tự publish generation và ranh giới visibility

Giao thức publish

Generation sử dụng snapshot_id ổn định. Ghi serving rows, history, quality và curated output bắt buộc; chờ ACK và xác minh logical counts. Sau khi outputs hoàn tất, manifest chuyển READY, rồi compare and set current pointer. API chỉ thấy thế hệ mới sau pointer swap. Writer crash trước bước này để API tiếp tục đọc generation cũ.

`source_order` là tuple feed_timestamp, original capture time và tie breaker ổn định. Writer so tuple với current, không dùng arrival order hoặc thời gian ghi Cassandra. Pointer CAS đọc pointer hiện tại, so source_order, cập nhật với điều kiện pointer chưa đổi; nếu CAS thất bại thì đọc lại và đánh giá lại. Late snapshot giữ audit/history nhưng không rollback pointer.

Generation READY bất biến. Retry tái sử dụng kết quả đã ready; không sửa payload, parent, expiry hoặc first visibility theo wall clock mới. Các feeds publish riêng nên response tổng hợp phải công bố mỗi snapshot version và as of; thiết kế không tạo atomic transaction xuyên tất cả feeds.

12 Snapshot semantics và phục hồi

Thay thế trạng thái theo feed

MVP hỗ trợ FULL DATASET theo default và schema GTFS Realtime đã pin; incrementality không hỗ trợ bị quarantine [S4]. Snapshot mới thay membership feed trong scope. Xe không còn trong membership được bỏ khỏi current view; điều này chỉ nói hệ thống không quan sát xe trong snapshot, chưa chứng minh xe đã kết thúc hoạt động.

Snapshot partial công bố is_partial cùng valid, invalid và excluded counts. Riêng alerts, sự vắng mặt trong snapshot partial không được dùng để đóng hàng loạt alerts cũ. Projection có thể giữ alert trước đó với trạng thái chưa xác nhận, nguồn gốc và thời hạn; chỉ snapshot đầy đủ đáng tin hoặc active period kết thúc mới cho phép kết luận tương ứng.

Observation regression và parent

Nếu xe vẫn thuộc membership nhưng observation_time lùi, ghi `OBSERVATION_TIME_REGRESSION`. Có thể kế thừa observation hợp lệ mới hơn từ generation trước cho xe đó; giữ timestamp gốc để tuổi tiếp tục tăng. Không mang xe đã vắng mặt sang danh sách fresh. Manifest lưu parent_generation_id và các inputs kế thừa cần cho retry; không chọn parent mới giữa hai lần thực hiện cùng generation.

Cam kết ghi và recovery

Tình huống	Hành vi thiết kế
Cùng snapshot hoặc observation được gửi lại	Upsert bằng khóa ổn định; READY generation được tái sử dụng; không counter increment
Crash sau một phần sink write	Pointer cũ giữ nguyên; retry cùng ID và parent hoàn tất outputs
Crash sau pointer swap trước checkpoint commit	Retry nhận generation READY; không tăng KPI hoặc tạo history trùng
Cassandra hay sink bắt buộc không ghi được	Throw lỗi để batch thất bại; không return thành công và bỏ dữ liệu
Lỗi một record	Quality và DLQ có reference; record disposition được hạch toán rõ
Checkpoint lạc sau Kafka retention	Phát hiện data loss; dừng có hướng dẫn replay hoặc run mới

Checkpoint dùng namespace pipeline_version/run_id/query_name trên persistent volume. `foreachBatch` có thể delivery lại tới sink ngoài; bảo đảm của MVP là at least once với idempotent writes, không phải exactly once transaction xuyên hệ thống [S5].

Curated batch path dựa trên run, query identity và batch_id; chỉ đọc batch có commit manifest. Retry không tạo tên ngẫu nhiên rồi để report đọc hai bản. Historical reports dedup observation_id trước aggregate. Không bắt buộc watermark cho query current snapshot; nếu thêm streaming window experiment, dùng query riêng và không tuyên bố watermark tự đưa mọi late row vào DLQ.

13 Mô hình Cassandra metadata và static

Thiết kế query first, denormalize theo đường đọc [S7–S8]. Ký hiệu `((a,b),c)` trong bảng nghĩa là partition key gồm a,b và clustering column c; không phải CQL đầy đủ. Những quan hệ logic GTFS không được thực hiện bằng full scan hoặc ALLOW FILTERING.

Bảng	Khóa	Nội dung và query
run_metadata	((run_id))	source_type, scope, gtfs_version, policy, application_clock, clock_sequence, run state; chi tiết một run
runs_by_project	((project_id), created_at, run_id)	Danh sách run theo thời gian có LIMIT và paging
feed_current	((run_id, feed_type))	current snapshot, source_order, first visibility, expires_at; đọc pointer một lần
snapshot_manifest	((run_id, feed_type), snapshot_id)	BUILDING hoặc READY, parent, counts, checksum, source_order, expiry và lineage
routes_by_version	((gtfs_version), route_id)	Danh mục route trong scope, tên và route_type
stops_by_version	((gtfs_version), stop_id)	Chi tiết trạm theo version và ID
stops_by_route	((gtfs_version, route_id), stop_id)	Tập trạm của một route; không cam kết stop order
stop_times_by_trip	((gtfs_version, trip_id), stop_sequence)	Stop occurrence, arrival và departure seconds theo trip
geometries_by_route	((gtfs_version, route_id), shape_id)	Geometry payload có bound cho từng shape của route

Quy tắc dữ liệu và truy vấn

API xác định gtfs_version từ run_metadata, không nhận một version tùy ý để nối dữ liệu. Static tables chỉ chứa dimensions cần cho scope và query ứng dụng. Service resolution đầy đủ được Spark thực hiện từ Parquet, không yêu cầu API join calendar, trips và stop_times trên Cassandra.

Clustering direction của runs_by_project có thể đặt created_at DESC cho lịch sử mới nhất. Payload geometry phải có giới hạn; nếu đo thấy shape quá lớn thì chia segment và cập nhật query mapping, schema, API và tests cùng nhau. Không tạo bảng lớn không có truy vấn sử dụng.

Metadata dùng text cho IDs và UTC timestamp cho wall instants. GTFS seconds dùng integer; optional values giữ null. Không TTL static version hoặc run metadata đang còn được reference. Retention version cũ phải kiểm tra các run và recording phụ thuộc trước khi xóa.

14 Mô hình Cassandra serving và retention

Bảng	Partition và clustering keys	Query và nội dung
vehicles_by_route_snapshot	((run_id, snapshot_id, route_id), vehicle_id)	Current membership, position, observation time, lineage và quality
vehicle_observations_by_day	((run_id, vehicle_id, utc_date), event_time, observation_id)	Lịch sử xe theo ngày UTC; time range có bound
predictions_by_stop_snapshot	((run_id, snapshot_id, stop_id), predicted_at, prediction_id)	Upcoming predictions, trip instance, event type, schedule resolution và quality
route_metrics_by_snapshot	((run_id, feed_type, snapshot_id), route_id)	KPI, metrics_as_of, target policy, numerator và denominator
alerts_by_route_snapshot	((run_id, snapshot_id, route_id), alert_id)	Alert đã resolve scope, active periods và inherited status
quality_by_snapshot	((run_id, feed_type, snapshot_id), reason_code)	Severity, disposition và counts có units
pipeline_status	((run_id), component)	Heartbeat, last success, batch progress và lỗi cuối

TTL và consistency

Current generation rows có TTL tham chiếu 3.600 giây; observation history TTL tham chiếu ba ngày với bucket UTC. Chốt expires_at theo wall clock lúc materialize đầu tiên; retry dùng thời gian còn lại, không kéo dài vòng đời bằng duplicate. Generation serving rows, manifest và pointer phải công bố expiry nhất quán; pointer hoặc audit metadata có thể giữ lâu hơn để giải thích generation đã hết hạn.

TTL tiếp tục chạy khi replay clock pause. Nếu generation hết hạn, API trả `generation_expired` với dữ liệu unavailable; không suy ra empty snapshot. Empty hợp lệ được chứng minh bằng READY manifest có expected counts bằng zero. Hết rows trước expiry do lỗi hoặc partial write là inconsistency, không được đổi thành zero xe.

Cassandra demo RF1 dùng consistency ONE hoặc LOCAL_ONE tùy driver và topology được pin; LWT metadata sử dụng serial consistency phù hợp cấu hình. RF1 không chống mất node. Production cần thiết kế topology, RF, read/write consistency và capacity riêng, không suy ra từ demo.

Prepared statements và bounded concurrency dùng cho sink. Không dùng Cassandra counters cho KPI, LWT mỗi observation hoặc logged batch qua hàng nghìn partitions như transaction toàn cục. Đo partition bytes và rows, tombstones, write latency trước khi tăng scope hoặc retention.

15 Định nghĩa vị trí và dự báo

Freshness của xe

Một xe fresh khi ID và position hợp lệ, observation timestamp đã biết, không vượt future tolerance và tuổi không quá 120 giây. Tuổi dùng application_clock. Unknown age trả null và nhãn unknown, không dùng header để giả observation mới.

Trường kết quả	Định nghĩa
fresh_vehicle_count	Unique vehicle IDs có position hợp lệ và tuổi trong ngưỡng
stale_vehicle_count	Unique vehicle IDs hợp lệ nhưng quá ngưỡng tuổi
unknown_age_vehicle_count	Position hợp lệ nhưng thiếu observation timestamp
invalid_vehicle_count	Entity xe bị loại theo policy; công bố đơn vị là entity
snapshot_membership_count	Unique vehicle IDs nhận diện được trong selected membership; không phải fleet count

Spark lưu metrics tại metrics_as_of. API đọc current rows có bound để đánh giá lại freshness tại request, kể cả khi feed không có input mới. Với multiple counts, không áp đặt tổng bằng membership nếu invalid entities thiếu ID; quality accounting giải thích đơn vị và overlap.

Provider adapter cho ETA

ETA lấy từ MBTA, chưa có model riêng. Adapter chỉ tính predicted delay khi schedule và cùng event type được resolve. Nếu nguồn chỉ có delay mà không có time, chỉ dựng predicted time khi có schedule đáng tin và relationship cho phép; không đủ dữ liệu thì unknown.

Bus uncertainty MBTA	Nhãn hiển thị	Policy KPI mặc định
Nhỏ hơn 300	Realtime prediction theo provider	Có thể eligible nếu các điều kiện khác đạt
Bằng 300	Schedule only	Loại khỏi realtime delay KPI
Bằng 301	Degraded prediction	Có thể hiển thị badge; loại khỏi KPI
Lớn hơn 301	Invalid prediction	Không hiển thị như ETA hợp lệ
Không có	Unknown uncertainty	Có thể eligible; công bố unknown_uncertainty_count

Mã 300 là dự báo dựa trên lịch mà MBTA có thể điều chỉnh bằng thời gian hành trình lịch sử, chưa phải realtime prediction. Các mã này là semantics riêng của MBTA bus [S2]. Prediction age threshold tham chiếu 90 giây; stale prediction có thể được trả để audit nhưng không hiển thị như dự báo hiện thời đủ chất lượng.

16 KPI độ trễ dự kiến và ví dụ

Định nghĩa tính toán

`predicted_delay_seconds = predicted_event_utc - scheduled_same_event_utc`. Delay âm là dự báo sớm, được giữ nguyên. Late threshold 300 giây là policy demo, không phải chuẩn on time chính thức của MBTA.

Mỗi trip instance chỉ chọn tối đa một target stop tương lai hợp lệ tại metrics_as_of, theo stop sequence và progress khi có. Cancelled, không resolve được lịch, target mơ hồ, schedule only, degraded hoặc invalid bị loại. Không tính KPI bằng cách coi từng stop prediction là một chuyến.

Trường	Công thức hoặc semantics
eligible_trip_count	Số trip instances có target prediction đạt policy
predicted_late_trip_count	Eligible trips có delay lớn hơn hoặc bằng 300 giây
predicted_late_rate	predicted_late_trip_count chia eligible_trip_count
mean_predicted_delay_seconds	Tổng delay của eligible trips chia eligible_trip_count
excluded_trip_count	Số trip instances bị loại, với reason breakdown

Nếu denominator bằng zero, rate và mean là null. Response luôn có policy version, metrics_as_of, target selection, counts và freshness. API đánh dấu stale khi clock tiến mà Trip Updates không cập nhật. Target và KPI giữ semantics as of generation, không được trình bày như đã tính lại target mới tại mọi request; ETA theo trạm có thể lọc thời điểm đã qua khi phục vụ.

Golden fixture và kết quả kỳ vọng

Fixture synthetic có agency timezone America/New_York, service date 2026 10 05 và application clock `2026-10-05T12:00:30Z`. IDs R_A, V1, V2, V3 và T1, T2 là IDs kiểm thử, chưa phải MBTA đã xác minh.

Đối tượng	Input fixture	Kết quả kỳ vọng
V1	Observation 12:00:10Z	Fresh, tuổi 20 giây
V2	Observation 11:57:00Z	Stale, tuổi 210 giây
V3	Position hợp lệ, timestamp thiếu	Unknown age, không fresh
T1	Scheduled 12:05Z, predicted 12:11Z	Delay 360 giây, late
T2	Scheduled 12:05Z, predicted 12:06Z	Delay 60 giây, chưa late

Khi Trip Updates còn fresh và quality đạt policy: fresh 1, stale 1, unknown 1; eligible 2, late 1, rate 0,5 và mean delay 210 giây. Đây là expected values dùng để kiểm thử, chưa phải test results đã chạy.

17 Thiết kế API và truy vấn

Tất cả endpoint dưới `/api/v1`, có OpenAPI và Pydantic models. run_id là bắt buộc cho endpoint dữ liệu, route hoặc stop phải thuộc scope. Metadata hỗ trợ chọn run có paging riêng.

GET endpoint	Mục đích và bảng đọc chính
/health/live và /health/ready	Liveness process; readiness dependencies, không đồng nghĩa feed fresh
/runs và /runs/{run_id}	runs_by_project, run_metadata; source, clock và scope
/routes	routes_by_version theo run static version
/routes/{route_id}/geometry	geometries_by_route, trả shapes riêng
/routes/{route_id}/stops	stops_by_route, tập stops không giả route order
/trips/{trip_id}/stops	stop_times_by_trip theo stop_sequence
/vehicles	feed_current và vehicles_by_route_snapshot
/vehicles/{vehicle_id}/history	vehicle_observations_by_day với time bounds
/stops/{stop_id}/predictions	feed_current và predictions_by_stop_snapshot
/routes/{route_id}/metrics	route_metrics_by_snapshot và bounded vehicle rows để refresh age
/alerts	feed_current và alerts_by_route_snapshot
/quality	quality_by_snapshot theo current feed pointers
/pipeline/status	pipeline_status và run metadata

Giới hạn và response timing

History mặc định tối đa 24 giờ, page size 100, max 500 là cấu hình tham chiếu. Query các day buckets UTC liên quan rồi merge có bound; paging token phải giữ run, vehicle, range và vị trí đọc nhất quán. Vehicles không chỉ định route chỉ được fan out các route đã cấu hình, tối đa năm ở MVP, với max_total_rows tham chiếu 2.000.

Mỗi request đọc pointer một lần cho mỗi feed rồi dùng cùng snapshot_id để lấy rows. Request tổng hợp giữ as of riêng cho positions, trip updates và alerts. Cache key phải bao gồm run và snapshot version; clock, expiry và freshness tại request không được bị cache che. Timeout cơ sở dữ liệu và API request phải hữu hạn.

API chỉ thực hiện bounded serving logic, ví dụ tính tuổi và gắn stale, không đọc raw để tự chạy lại join hoặc xây KPI thay Spark. Khi query vượt giới hạn, trả validation error có hướng dẫn thu hẹp range hoặc dùng paging.

18 Response contract và lỗi

Envelope response

Mọi response dữ liệu có `data`, `meta` và `warnings`. Thời gian API dùng ISO 8601 UTC có Z; các duration dùng seconds, field names ghi đơn vị. Không lẫn epoch milliseconds và seconds.

```
{
  "data": {"fresh_vehicle_count": 1, "stale_vehicle_count": 1},
  "meta": {
    "run_id": "fixture-run",
    "source_type": "synthetic",
    "gtfs_version": "fixture-static-v1",
    "application_clock": "2026-10-05T12:00:30Z",
    "served_at": "2026-10-05T15:00:00Z",
    "feeds": {
      "vehicle_positions": {
        "snapshot_id": "fixture-snapshot-1",
        "metrics_as_of": "2026-10-05T12:00:30Z",
        "is_partial": false,
        "state": "available"
      }
    },
    "coverage": {"scope_route_count": 1, "denominator_unit": "vehicle_id"}
  },
  "warnings": []
}
```

Ví dụ trên chỉ minh họa contract synthetic. Payload chính thức còn cần policy, expiry, observation origin và chất lượng tùy endpoint. KPI prediction phải có numerator và denominator, không chỉ phần trăm.

HTTP status	Điều kiện	Contract
200	Dữ liệu hợp lệ hoặc stale vẫn hữu ích	data cùng state, age, warnings và as of rõ ràng
200	Empty snapshot hoặc không có prediction hợp lệ	Tập rỗng và reason; không tự tạo ETA
404	Run hoặc entity ngoài phạm vi truy cập không tồn tại	Error code và reference; không trả zero KPI
422	Params, range hoặc paging không hợp lệ	Validation details có thể sửa
503	Cassandra unavailable, generation expired hoặc inconsistent	code rõ lý do, data unavailable; không giả empty

Error envelope chứa `error.code`, `message`, `request_id`, `retryable` và metadata run nếu có. UI có trạng thái loading, empty, stale, expired và error riêng. API readiness không coi nguồn MBTA timeout là Cassandra mất sẵn sàng; source health được trả ở run và pipeline status.

19 Thiết kế dashboard

Streamlit gọi FastAPI; không kết nối Cassandra hoặc MBTA trực tiếp. Banner trên mọi trang có run_id ngắn, LIVE hoặc REPLAY hoặc SYNTHETIC, scope, clock nghiệp vụ và served_at. Replay paused hoặc finished vẫn giữ historical clock, không tự chuyển sang live.

Trang	Nội dung và thao tác	Trạng thái cần hiển thị
Tổng quan tuyến	Chọn route; fresh, stale, unknown; predicted late rate cùng denominator; feed status	null KPI, stale metrics, partial coverage
Bản đồ	Shapes và stops thật; xe bằng marker; tooltip ID, observation time, tuổi và quality	Màu fresh, stale, unknown; route có nhiều shapes
Trạm và dự báo	Chọn stop, route và direction; arrival hoặc departure, delay nếu đủ điều kiện	Schedule only, degraded, unknown; không có ETA
Cảnh báo	Active periods, nội dung, selectors đã resolve; lọc route	MBTA service alerts tách pipeline warnings
Chất lượng và pipeline	Reason counts, source age, input units, generation, offsets, measured latency	Source outage, sink outage, paused replay, expiry

Cách trình bày KPI

Thẻ tỷ lệ chậm ghi “Độ trễ dự kiến” và “1 trên 2 chuyến đủ điều kiện”, kèm as of. Denominator bằng zero hiển thị “Chưa có mẫu đủ điều kiện”, không hiển thị 0 phần trăm. Số xe ghi “Xe có vị trí mới”, không gọi là toàn bộ xe đang hoạt động của MBTA.

Chi tiết xe mở lịch sử theo UTC range, bảng observation và đường quan sát. Thiếu speed optional không làm mất marker. Derived speed, nếu bổ sung, được ghi là ước lượng trung bình đoạn quan sát; không diễn giải thành mức tắc nghẽn đường.

Refresh và offline

Refresh mặc định 5 giây theo cơ chế của ứng dụng; tránh busy loop hoặc reload khóa UI. Lỗi một API panel không xóa nhãn nguồn và các panel còn hữu ích. Khi stale, giữ giá trị cũ có thời gian và badge; khi expired, hiện unavailable.

PyDeck hiển thị PathLayer và ScatterplotLayer từ shapes, stops và coordinates. Core demo không yêu cầu Mapbox token. Offline dùng nền không tile hoặc nền đã chuẩn bị hợp pháp, đồng thời phải thử JavaScript assets và dependencies thực tế; bỏ basemap chưa chứng minh dashboard chạy offline.

20 Chất lượng dữ liệu và observability

Nhóm reason	Ví dụ	Disposition mặc định
Contract hoặc transport	INVALID_JSON, INVALID_HEADER, UNSUPPORTED_SCHEMA, PAYLOAD_TOO_LARGE	Quarantined; không publish state giả
Xe	MISSING_VEHICLE_ID, INVALID_COORDINATES, FUTURE_TIMESTAMP	Record rejected hoặc excluded from freshness
Thời gian và lịch	MISSING_OBSERVATION_TIMESTAMP, AMBIGUOUS_SERVICE_DATE, UNRESOLVED_STOP_SEQUENCE	Retained có warning hoặc excluded from KPI
Prediction	NO_PREDICTION_DATA, SCHEDULE_ONLY_PREDICTION, DEGRADED_PREDICTION, INVALID_PREDICTION	Hiển thị theo policy; không lẫn vào eligible KPI
Lineage và ordering	STATIC_VERSION_MISMATCH, DUPLICATE_OBSERVATION, LATE_SNAPSHOT_NOT_PUBLISHED	Mismatch quarantined; duplicate hoặc late có audit
Projection	OBSERVATION_TIME_REGRESSION, UNKNOWN_ROUTE_OR_TRIP	Inherited hoặc excluded theo policy, có lineage

Reason có severity error, warning hoặc info và disposition độc lập. Một record có thể có nhiều warnings; không cộng warning counts để suy ra số record lỗi. DLQ chứa error_id, run/feed/snapshot/entity reference, reason, schema, wall time, excerpt có giới hạn và raw reference.

Hạch toán đơn vị

```
selected_entities = fully_valid + partially_valid + rejected_entities
candidate_predictions = retained + rejected + not_applicable_predictions
valid_candidate_observations = unique + duplicate_observations
```

Trip Update có thể explode thành nhiều predictions. Kafka message count, source entity count, normalized candidate count và unique record count là các đơn vị khác nhau. Scope excluded entities được hạch toán trước selected_entities và giữ trong capture metadata.

Metrics và logs

Metrics tối thiểu gồm HTTP attempts và bytes, outbox pending, disk usage, Kafka bytes/messages, normalized/unique/duplicate counts, Spark batch duration, sink errors, write latency, first visibility latency và API latency. JSON logs có component, run, feed, snapshot, batch, stage, status, duration và reason. Duration nội bộ dùng monotonic timer.

Kafka backlog của Spark so broker end offsets với query progress hoặc checkpoint offsets tương ứng, theo topic và partition; không mặc định consumer group committed offsets là checkpoint của Spark [S6]. `pipeline_visibility_latency` lấy first successful view publication trừ enqueue đầu tiên của logical generation; duplicate có metric riêng, không tạo latency âm bằng cách dùng enqueue của retry.

21 Replay synthetic và demo offline

Replay recording thật

Recording manifest giữ static version, scope, capture order, checksums và cả ba feeds. Replay tạo run mới, đọc đúng static và raw, không gọi HTTP. Source times giữ nguyên; wall published_at và processed_at thuộc lần replay hiện tại. Khoảng chờ lấy capture timeline chia speed; positions, updates và alerts được interleave trên cùng timeline.

Control topic phát clock_tick có clock_sequence tăng. Processor chỉ advance clock theo tick mới hợp lệ, kể cả qua đoạn không có dữ liệu để thử stale. Prediction arrival time không được dùng làm clock. Pause đóng băng clock nghiệp vụ; heartbeat wall time vẫn hoạt động. Finish giữ clock cuối và state finished.

Synthetic scenarios

Fixture generator seed cố định tạo static và realtime nhất quán, source_type synthetic và scenario_id. Có duplicate snapshot, observation lặp, late snapshot, invalid GPS, missing timestamp, cancelled/degraded prediction, empty feed, malformed response, dừng một feed và crash trước pointer. Synthetic không tự thay live khi nguồn MBTA lỗi.

Stress test có thể nhân patterns để đạt 100, 500 hoặc 1.000 normalized candidate records mỗi giây, nhưng phải giữ references và trip instances đúng. Báo cáo ghi rõ phép biến đổi; không gọi các bản sao này là số xe MBTA thật.

Offline checklist có thể kiểm chứng

Điều kiện	Bằng chứng cần lưu
Container images và dependencies đầy đủ	Versions manifest; chạy không kéo image từ mạng
Python wheels và Spark Kafka JARs đã có	Lock files và connector list; không tải lúc demo
Static và recording checksum khớp	Preflight report; fail nếu thiếu đúng version
UI assets không cần CDN ngoài	Kiểm tra mạng tắt và screenshot các trang
Clock và TTL hoạt động đúng	Pause test, stale test và generation expiry test

Docker chưa có hoặc daemon không được phép là prerequisite cần chuẩn bị trên máy được phép. Unit tests chạy độc lập chưa phải nghiệm thu Kafka Spark Cassandra. Không đổi deployment ngầm hoặc bỏ component để ghi là offline demo đã thành công.

22 Triển khai cấu hình và an toàn

Bộ phiên bản tham chiếu

Python 3.11, Java 17, Spark 3.5.7 với Scala 2.12 là bộ tham chiếu tương thích cho thiết kế [S5–S6]. Kafka, Cassandra, driver, FastAPI và Streamlit được pin tag hoặc version sau preflight. Không dùng tag latest. File versions.md ghi image digests, connector artifact, Python locks và kết quả smoke test. Đây chưa phải danh sách dependencies đã được chạy xác nhận.

Cấu hình	Giá trị tham chiếu và ý nghĩa
SELECTED_ROUTE_IDS	Năm IDs đã xác minh trong ZIP; bắt buộc, không random fallback
POLL_INTERVAL_SECONDS và SPARK_TRIGGER_SECONDS	30 giây mỗi feed và 5 giây micro batch
VEHICLE_STALE_SECONDS và PREDICTION_STALE_SECONDS	120 giây và 90 giây; policy project
PREDICTED_LATE_SECONDS và FUTURE_TOLERANCE_SECONDS	300 giây và 5 giây
MAX_SNAPSHOT_BYTES	4 MiB envelope serialized; phối hợp transport limits
RAW_DISK_BUDGET_BYTES và KAFKA_RETENTION_HOURS	2 GiB tham chiếu và 6 giờ mục tiêu
CURRENT_GENERATION_TTL và HISTORY_TTL	3.600 giây và 3 ngày wall clock
CHECKPOINT_ROOT và RAW_ROOT	Persistent volumes riêng, namespace theo run
API_HISTORY_MAX_HOURS và API_PAGE_SIZE_MAX	24 giờ và 500 rows; bound fan out

Đơn vị deploy

Docker Compose dự kiến gồm Kafka, Cassandra, init jobs, collector, Spark processor, FastAPI và dashboard; batch analytics là job theo lệnh. Static loader hoàn tất trước processor. Tách volumes raw, checkpoint, Cassandra data và curated. Healthchecks cùng application readiness quyết định khởi động; depends_on đơn thuần không chứng minh dependencies sẵn sàng.

Host ports bind 127.0.0.1; Kafka và Cassandra chỉ giao tiếp trong mạng Compose. CORS theo allowlist dashboard. Demo local chưa có cơ chế auth production; nếu public API cần bổ sung auth, rate limits và review trước triển khai công khai. Secrets hoặc tokens không đưa vào repo hoặc logs. HTTP public MBTA không được gắn credential giả.

Không chạy sudo, sửa dịch vụ host hoặc xóa volumes để chữa lỗi tự động. Các thao tác reset checkpoint hay xóa dữ liệu cần runbook rõ phạm vi, vì chúng có thể làm mất khả năng phục hồi đúng của run.

23 Runbook vận hành và xử lý sự cố

Khởi động và dừng có kiểm soát

1. Preflight runtime, disk, versions và inputs; chốt static version và scope đã validate.
2. Khởi động Kafka và Cassandra; chạy topic init, CQL migrations và connectivity checks.
3. Tạo run_metadata, mở processor cùng checkpoint namespace; chờ query ready rồi chạy collector hoặc replay.
4. Khởi động API và dashboard; xác minh READY generation, counts, source banner và quality.
5. Khi dừng, ngừng publisher trước, chờ outbox và Spark backlog trong giới hạn; lưu progress và manifests, rồi dừng services mà giữ volumes.

Hiện tượng	Kiểm tra và hành động	Điều không được che
Một feed timeout	HTTP status, retries, last successful header; giữ view có stale badge	Không publish empty để reset state
Outbox tăng	Kafka connectivity, ACK, payload bytes, broker budget; retry cùng IDs	Không mark published trước ACK
Raw disk gần đầy	Disk metrics, recording rate, budget; dừng capture nếu không lưu bền được	Không tiếp tục publish rồi nhận có audit đầy đủ
Spark batch fail	Reason, checkpoint offsets, sink health; sửa lỗi rồi restart cùng checkpoint	Không nuốt exception hoặc tạo checkpoint random
Cassandra unavailable	Node và driver errors; phục hồi service, retry batch	Không trả zero KPI thay 503
Current generation expired	expires_at và row availability; báo unavailable, chạy tiếp hoặc run mới	Không kéo dài TTL bằng duplicate ngầm
Kafka offsets đã mất	Retention, checkpoint, raw manifest; replay vào run mới	Không tắt failOnDataLoss để báo thành công
Replay thiếu static	Verify checksum và artifact đúng version	Không join với ZIP live mới tùy ý

Kiểm tra sau phục hồi

Đối chiếu current pointer, READY manifest, expected row counts và unique observation keys. Kiểm tra batch progress đã tăng, input backlog giảm, API source và clock vẫn đúng run. Ghi thời gian từ restart tới generation đủ điều kiện đầu tiên cùng fault scenario; không tuyên bố recovery thành công chỉ vì process đã lên.

Repository cần hiện thực các lệnh preflight, init, ingest, live, record, replay, test và benchmark; tên lệnh và hướng dẫn được bổ sung khi triển khai.

24 Kế hoạch kiểm thử và oracle

Unit tests

Kiểm tra canonical hash và IDs; parse optional fields; source enum/defaults; coordinates và future time; calendar exception, giờ 25:10, DST; trip instance ambiguity; repeated stop resolution; uncertainty adapter; target selection và zero denominator; alert selector predicate; freshness và run isolation. Oracle cần được tính độc lập trên fixture nhỏ, không gọi lại chính hàm triển khai để tự chứng minh.

Integration scenarios bắt buộc

Mã	Scenario	Kết quả cần chứng minh
IT01	Golden fixture ban đầu	Counts, rate 0,5 và mean 210 giây đúng ở API và sink
IT02	Snapshot mới phát hai lần	Membership không tăng; history cùng observation không trùng
IT03	Snapshot cũ tới sau snapshot mới	Pointer và vị trí không rollback; late có audit
IT04	Clock tiến, không có positions mới	V1 tuổi 135 giây, fresh 0; predictions cũ báo stale
IT05	Empty hợp lệ so malformed hoặc timeout	Empty thay view; malformed giữ view cũ có source warning
IT06	Missing timestamp và GPS invalid	Unknown không fresh; invalid disposition và counts đúng
IT07	Ambiguous date và repeated stop	Không đoán lịch hoặc stop; loại KPI có reason
IT08	Bus uncertainty và cancelled trip	300, 301, lớn hơn 301, absent xử lý đúng policy
IT09	Partial alerts	Không đóng hàng loạt alert cũ; inherited lineage đúng
IT10	Crash partial write trước READY hoặc CAS	API vẫn thấy generation cũ; retry hoàn tất cùng IDs
IT11	Crash sau CAS trước checkpoint commit	Reuse READY; first visibility và TTL không đổi
IT12	Cassandra hoặc required sink down	Batch fail, offsets không được coi là xử lý thành công
IT13	Message gần 4 MiB và vượt ngưỡng	Transport boundary đạt; oversize lưu raw và reject có reason
IT14	Run isolation và replay pause	Output tách run; clock freeze, wall heartbeat tiếp tục
IT15	Generation TTL expiry	503 generation_expired; phân biệt zero rows hợp lệ
IT16	Demo mạng tắt	Pipeline, API, assets và bản đồ core dùng được

Snapshot thứ hai của oracle lúc 12:01Z giữ V1 observation 12:00:55Z, V3 còn, V2 vắng. Clock 12:03:10Z làm V1 tuổi 135 giây. Test reports lưu input checksum, run, versions, expected, actual và status. Toàn bộ status hiện tại là **chưa thực hiện**.

25 Benchmark và bằng chứng thành công

Thiết kế thí nghiệm

Benchmark dùng recording thật replay 1x, 10x, 50x để kiểm tra timeline và source characteristics. Synthetic stress dùng 100, 500 và 1.000 normalized candidate records mỗi giây; snapshot rate, records per snapshot và bytes phải được công bố riêng. Không tăng HTTP polling MBTA để tạo tải.

Mỗi cấu hình có warm up 60 giây, đo 120 giây, ba lần chạy nếu tài nguyên cho phép. Ghi CPU, RAM, disk, versions, routes, input checksums, scenario seed, tốc độ phát và clock assumptions. Nếu không đạt đủ protocol, ghi thời lượng và số lần thực tế; không tự điền kết quả chưa đo.

Metric	Đơn vị và cách đo
Input throughput	Snapshots mỗi giây, bytes mỗi giây và normalized candidates mỗi giây
Output throughput	Unique retained records mỗi giây; duplicate rate có denominator
Visibility latency	p50, p95, p99 của first view publication trừ first enqueue wall time
Backlog	End offset trừ processed progress theo topic và partition
Processing	Batch duration, records processed và sink write latency
Resource usage	CPU, RSS RAM, disk bytes và partition size thực đo
Recovery time	Từ restart tới generation đạt yêu cầu sau fault; so oracle và duplicates
Source age	Application clock trừ observation hoặc header; không gọi là transport latency

Kết luận được phép rút ra

Thành công cần đồng thời có dữ liệu phục vụ hữu ích, đúng fixture, biến đổi Spark quan sát được, Cassandra query tables phù hợp, idempotency và recovery trong fault tests, cùng benchmark có số liệu thực tế. Không chỉ chứng minh có message đi từ producer tới consumer.

Target p95 dưới 10 giây ở baseline 4 CPU và khoảng 16 GiB RAM là mục tiêu chưa đo. Nếu vượt, báo measured value, backlog và giới hạn tải; giảm scope hoặc tối ưu có bằng chứng rồi chạy lại các checks liên quan. Không viết hệ thống đạt thời gian thực chỉ từ trigger 5 giây.

Artifact bằng chứng

`benchmark_results.csv`, `benchmark_report.md`, `test_report.json`, manifests raw/curated, source_probe_report, static_quality_report, query progress và API samples được lưu theo run. Screenshot và video demo bổ trợ bằng chứng, không thay số liệu hoặc oracle. Log phải đủ truy vết mà không chứa secret hoặc body lớn không cần thiết.

26 Nghiệm thu và truy vết yêu cầu

Yêu cầu	Thành phần chịu trách nhiệm	Kiểm chứng chính
FR01	Static loader, dimensions và static catalog	Static checks, unit calendar/DST, IT07
FR02 và FR03	Collector, raw archive, outbox, Kafka	Raw checksum, outbox restart, IT05 và IT13
FR04	Spark normalization, join và quality	DataFrame plan, normalized output, unit validation
FR05	Position projection và API freshness	IT01, IT02, IT04 và IT06
FR06 và FR07	Schedule resolver, prediction adapter và KPI	Golden delay values, IT07 và IT08
FR08	Alert selectors và projection	Selector tests và IT09
FR09	Observation sink, curated và Spark batch	History dedup, committed batches và report lineage
FR10	Manifest, READY protocol và pointer CAS	IT02, IT03, IT10, IT11 và IT15
FR11	Replay publisher, control clock, run namespace	IT04, IT14 và IT16
FR12	API, dashboard, tests và evidence pack	Contract tests, demo và benchmark report
NFR01 và NFR02	Oracle, stable keys và persistent checkpoints	IT01–IT12 với expected và actual outputs
NFR03 và NFR04	End to end instrumentation, budgets	Benchmark percentiles, size boundary và disk measurements
NFR05 đến NFR08	Lineage, dependency locks, metrics và query bounds	Metadata audit, explain query mapping, paging tests
NFR09 và NFR10	Deploy configuration và offline assets	Local binding/CORS inspection và IT16

Điều kiện chấp nhận MVP

MVP được chấp nhận khi cả ba phần môn học có bằng chứng: nghiệp vụ có câu hỏi và dữ liệu phù hợp; data engineering dùng thật Spark và Cassandra; ứng dụng chỉ dùng serving pipeline. Required golden và integration cases phải được chạy, reports ghi phiên bản và kết quả thực tế. Benchmark được trình bày với target và measured results tách biệt.

Demo khoảng mười phút gồm chọn run và source, bản đồ và ETA, đối chiếu golden KPI, theo dõi quality, phát duplicate/late, crash/restart và replay khi mạng tắt. Chỉ claim các lỗi đã thử và phạm vi đã đo; RF1 chưa có HA và chưa có ground truth để đánh giá ETA accuracy.

Trước khi nộp, bổ sung route IDs đã chốt, actual dependency versions, instructions đã chạy được, known issues và bảng test status. Tài liệu hiện tại là baseline để nghiệm thu; mọi mục chưa có bằng chứng vẫn được đánh dấu chưa thực hiện.

27 Tổ chức triển khai và bàn giao

Giai đoạn	Công việc	Điều kiện kết thúc
A Nền tảng	Source probe, static schema, scope, versions, Compose và topic/migration init	GTFS version hợp lệ, samples có checksum, dependencies kết nối được
B Vị trí xe	Collector, Kafka, Spark, generation, position API và map	Vertical slice hoàn chỉnh; freshness, duplicate và late tests đạt
C Predictions và alerts	Calendar/stop resolution, provider adapter, target KPI và selectors	Golden delay và uncertainty tests đạt; UI badges đúng
D Reliability và replay	Durable outbox, checkpoint recovery, immutable outputs, control clock	Fault scenarios có oracle; run isolation và offline test đạt
E Batch và đo lường	Curated historical reports, instrumentation và benchmark	Báo cáo có nguồn, units, actual values và giới hạn
F Bàn giao	README, runbook, tài liệu, evidence pack và demo	Máy mới chuẩn bị đủ dependencies chạy theo hướng dẫn

Phân công tham chiếu

Nhóm ba người có thể phân công người thứ nhất phụ trách nguồn, static và collector; người thứ hai Spark, Cassandra và recovery; người thứ ba API, dashboard và evidence. Schema, IDs, thời gian và golden oracle phải thống nhất trước khi làm song song. API và UI có thể phát triển bằng fixture có nhãn synthetic, nhưng nghiệm thu cuối phải qua pipeline đầy đủ.

Cấu trúc repository dự kiến

Thư mục	Nội dung
collector và replay	HTTP ingestion, raw/outbox, timeline và control publisher
pipelines và static	Spark streaming/batch, GTFS loader và dimensions
api và dashboard	FastAPI models/routes, Streamlit và assets
schemas và migrations	Envelope schemas, reason enums, CQL và version migrations
tests và fixtures	Unit/integration, oracle, synthetic GTFS và scenarios
config và deploy	Route scope, policy, Compose, locks và init scripts
docs và reports	System design, runbook, measured reports và demo evidence

Data volumes, raw recordings và checkpoint không commit vào Git như code. Repository phải mô tả cách chuẩn bị artifacts và kiểm tra checksum. Thay đổi schema_version, scope policy hay time semantics cần cập nhật fixture, CQL, API, documentation và test expectations nhất quán.

28 Phụ lục CQL và state machines

Minh họa CQL cho pointer và serving view

Các đoạn dưới là ví dụ cấu trúc lõi, chưa phải toàn bộ migrations. `source_order` được encode canonical có thứ tự so sánh nhất quán; implementation vẫn so tuple và CAS theo pointer đã đọc, không dùng timestamp ghi làm thứ tự nguồn.

```
CREATE TABLE IF NOT EXISTS feed_current (
  run_id text,
  feed_type text,
  snapshot_id text,
  source_order text,
  view_published_at timestamp,
  expires_at timestamp,
  PRIMARY KEY ((run_id, feed_type))
);

CREATE TABLE IF NOT EXISTS vehicles_by_route_snapshot (
  run_id text,
  snapshot_id text,
  route_id text,
  vehicle_id text,
  observation_id text,
  observation_time timestamp,
  latitude double,
  longitude double,
  freshness_known boolean,
  quality_flags set<text>,
  PRIMARY KEY ((run_id, snapshot_id, route_id), vehicle_id)
);

UPDATE feed_current
SET snapshot_id = ?, source_order = ?,
    view_published_at = ?, expires_at = ?
WHERE run_id = ? AND feed_type = ?
IF snapshot_id = ?;
```

Bootstrap pointer dùng INSERT IF NOT EXISTS. Trước update, writer so source_order mới với order hiện tại, chỉ CAS khi mới hơn; nếu pointer đã bằng cùng generation thì coi là idempotent success. Prepared statement inserts serving rows kèm TTL còn lại theo expires_at; không dùng counter cho counts.

State machines

Run đi từ CREATED tới RUNNING, có PAUSED cho replay, rồi FINISHED hoặc FAILED. Resume PAUSED giữ clock sequence; restart FAILED cần cùng run/checkpoint nếu inputs còn phục hồi được. Generation đi từ BUILDING tới READY; visibility chỉ thay khi pointer được publish. Late READY có thể chưa từng thành current. Expiry là trạng thái availability dựa vào expires_at, không biến READY thành empty generation.

29 Nguồn tham chiếu và thuật ngữ

Nguồn chính thức

Mã	Tài liệu và phạm vi sử dụng
S1	MBTA GTFS Static Documentation — trường và đặc thù static
S2	MBTA GTFS Realtime Documentation — feeds và bus uncertainty
S3	GTFS Schedule Reference — calendar, service time, stop_times và timezone
S4	GTFS Realtime Reference — header, entities, descriptors và optional fields
S5	Spark 3.5.7 Structured Streaming Guide — checkpoint, foreachBatch và watermark
S6	Spark 3.5.7 Kafka Integration — offsets, options và group ID
S7	Cassandra Logical Data Modeling — query first và denormalization
S8	Cassandra CQL Data Manipulation — writes, TTL và conditional operations

Các nguồn được dùng để đối chiếu thiết kế ngày 05 tháng 10 năm 2026. Mốc polling, stale, late, TTL, resource budget và target latency trong tài liệu là policy dự án; không phải cam kết của MBTA hoặc chuẩn mặc định của công nghệ.

Thuật ngữ

Thuật ngữ	Nghĩa trong tài liệu
Run	Một lần vận hành có nguồn, scope, static version và clock xác định
Snapshot	Trạng thái feed được capture và selected theo scope
Generation	Bộ outputs bất biến của snapshot, có manifest và điều kiện READY
Current pointer	Metadata chỉ generation được công bố để API đọc
Observation	Một lần đo nguồn hoặc state sample có lineage rõ
Trip instance	Một chuyến theo descriptor và service date, không chỉ trip_id
Predicted delay	Chênh lệch giờ dự báo với cùng sự kiện trong lịch
DLQ	Kênh record lỗi có reason và raw reference để điều tra
Lineage	Quan hệ từ output tới run, version, snapshot và raw
Application clock	Đồng hồ nghiệp vụ dùng cho freshness và tương lai của ETA

Tài liệu cần được cập nhật sau triển khai bằng phiên bản chạy thực tế, measured results và biên bản test. Bản thiết kế này không thay thế bằng chứng vận hành.